

Coach'In

Audit Technique

1. Introduction et contexte

Coach'In est une plateforme numerique de mise en relation entre sportifs et coachs certifies. En tant qu'intermediaire technique au sens de la loi LCEN, elle ne fournit pas directement de prestations de coaching mais cree l'environnement permettant cette relation. Le present audit couvre l'ensemble des aspects techniques du projet.

Objectifs de cet audit :

- Evaluer la solidite de l'architecture technique choisie
- Identifier les risques de securite et proposer des contre-mesures concretes
- Verifier la conformite WCAG 2.1 AA et RGAA 4.1
- Analyser l'environnement d'execution et ses contraintes operationnelles
- Definir les voies d'evolution et de migration technique

1.1 Stack technique retenue

Couche	Technologie	Justification technique
Frontend Web	Next.js 14 (React)	SSR natif, SEO optimal, React Aria accessibilite, App Router 14
Frontend Mobile	Flutter (Dart)	Codebase unique iOS/Android, widget Semantics, TalkBack/VoiceOver natifs
Backend API	NestJS (Node.js)	Architecture modulaire, Guards JWT/roles, validation stricte, TypeScript natif
Base de donnees	PostgreSQL + TypeORM	ACID, requetes complexes, migrations versionnees, typage TypeScript
Infrastructure	Docker Compose + CI/CD	Environnements isoles dev/staging/prod, pipelines automatises, HTTPS SSL/TLS

2. Architecture technique

2.1 Vue d'ensemble et flux de donnees

L'architecture suit un modele client-serveur en trois couches strictement separees. Cette separation garantit la maintenabilite, la securite par isolation et la scalabilite independante de chaque composant.

Flux : Client (Next.js / Flutter) --> HTTPS/TLS --> NestJS REST API --> PostgreSQL. Les cookies JWT httpOnly transitent uniquement via le serveur Next.js, jamais exposes au JavaScript client.

2.1.1 Frontend Web - Next.js 14 App Router

Le frontend web utilise l'App Router avec rendu hybride SSR/CSR. Les pages publiques (catalogue coaches, profils) sont rendues cote serveur pour le SEO. Les composants interactifs (dashboards, messagerie) utilisent des Client Components.

- Pages SSR : /coachs, /coachinfo/[id], page d'accueil
- Pages CSR : /profilsportif, /profilcoach, /admin
- Route API Next.js : /api/auth/set-token pour la gestion des cookies httpOnly
- Variable NEXT_PUBLIC_API_URL : URL publique backend uniquement, jamais de credentials

Point d'attention : La route GET /api/auth/set-token retourne le token JWT en clair dans la reponse JSON. Pour maximiser la securite, les appels backend sensibles devraient etre proxies cote serveur Next.js, sans re-exposer le token au client JavaScript.

2.1.2 Backend API - NestJS

NestJS structure l'API en modules independants avec injection de dependances. Chaque domaine metier constitue un module autonome.

- Module Auth : JWT strategy, Guards authentication, gestion sessions
- Module Users : inscription, profils, roles (athlete / coach / admin)
- Module CoachDocuments : upload S3, workflow approve/refuse admin
- Module Relationships : demandes de suivi athlete -> coach, statuts
- Guards roles : @Roles('athlete'), @Roles('coach'), @Roles('admin')
- Pipes de validation : class-validator sur tous les DTOs entrants

2.1.3 Authentification JWT httpOnly

Le stockage du JWT en cookie httpOnly plutot qu'en localStorage est une decision de securite fondamentale contre les attaques XSS.

Flux auth : POST /api/auth/login --> NestJS retourne { token } --> Next.js route /api/auth/set-token stocke le cookie httpOnly (7j) --> GET /api/auth/set-token retourne { authenticated, token } --> Authorization: Bearer pour les appels backend.

Securite : Le cookie httpOnly est inaccessible depuis JavaScript, bloquant toute exfiltration via XSS, meme si du code malveillant est injecte dans la page.

2.2 Schema de base de donnees

Entite	Champs cles	Relations
User	id (UUID), email, username, password_hash, role	OneToOne -> AthleteProfile / CoachProfile
AthleteProfile	id (UUID), age, goals, tags[]	ManyToMany -> Tags, OneToMany -> Relationships
CoachProfile	id (UUID), specialty, bio, isApproved	ManyToMany -> Tags, OneToMany -> CoachDocuments
CoachDocument	id (UUID), fileUrl (S3), isApproved, refusedAt	ManyToOne -> CoachProfile
Relationship	id (UUID), athleteId, coachId, status, createdAt	ManyToOne -> Athlete, ManyToOne -> Coach
Tag	id (UUID), name, categoryId	ManyToOne -> Category, ManyToMany -> Profiles

Mots de passe : Les mots de passe doivent etre hashesd avec bcrypt (salt rounds >= 12). Ne jamais stocker ni logger de mots de passe en clair. Verifier que les migrations ne contiennent pas de seeds avec credentials.

3. Audit de securite - OWASP Top 10

3.1 Analyse des menaces principales

A01 - Controle d'accès défaillant

Risque : Eleve	Un athlete pourrait acceder aux donnees d'un autre athlete ou modifier un profil coach.
Contre-mesure	Guards NestJS verifiant le role ET l'identite du token JWT. Route /athletes/me retourne uniquement les donnees du porteur du token. Aucun ID utilisateur accepte en body sans verification croisee avec le token.

A02 - Echecs cryptographiques

Risque : Moyen	Exposition de donnees sensibles (mots de passe, tokens) si le chiffrement est insuffisant.
Contre-mesure	HTTPS obligatoire (TLS 1.3). Bcrypt salt ≥ 12 pour les mots de passe. JWT cookie httpOnly + secure + sameSite:lax. Connexions PostgreSQL via SSL. Fichiers diplomes S3 avec URL pre-signees a expiration courte.

A03 - Injection SQL

Risque : Faible	TypeORM utilise des requetes parametrees par default, eliminant le risque d'injection directe. Validation class-validator (@IsString, @IsEmail, @IsUUID) sur tous les DTOs NestJS.
------------------------	--

A07 - Cross-Site Scripting (XSS)

Risque : Moyen	Injection de code malveillant dans les bios coaches, messages ou objectifs athletes.
Contre-mesure	React echappe automatiquement les contenus JSX. CSP headers configures dans Next.js. Sanitisation cote backend avant persistence. Cookie httpOnly immunise contre l'exfiltration de token via XSS.

A05 - Mauvaise configuration de securite

Risque identifie : Variables d'environnement potentiellement exposees, swagger public en production, logs verbeux avec stack traces, headers securite manquants.

Contre-mesures requises :

- Headers HTTP : X-Frame-Options DENY, X-Content-Type-Options nosniff, HSTS, CSP strict
- Swagger UI desactive en production (NODE_ENV === 'production')
- Logs production : niveau error uniquement, sans stack traces exposees
- Fichiers .env non commites - utilisation de secrets managers (AWS SSM, Vault)
- Separation stricte des configurations Docker par environnement

3.2 Gestion des roles et permissions

Role	Acces autorises	Acces interdits
ATHLETE	Son profil, ses tags, ses programmes, ses relations, profils coachs publics	Profils autres athletes, documents diplomes, panel admin
COACH	Son profil, ses programmes, ses relations acceptees, ses documents	Donnees athletes non relies, panel admin, profils autres coachs (prives)
ADMIN	Documents en attente, approbation/refus coachs, statistiques globales	Modification des donnees metier (profils, programmes, messages)

CRITIQUE - Admin sans auth : Les routes /coach-documents/* sont actuellement accessibles sans authentication. Ajouter @UseGuards(JwtAuthGuard, RolesGuard) @Roles('admin') immediatement. C'est le risque le plus urgent du projet.

3.3 Conformite RGPD

Obligation RGPD	Statut	Implementation
Consentement explicite a l'inscription	Operationnel	Checkbox CGU lors du signup
Privacy by design	Operationnel	Acces limite par Guards, separation roles
Audit logs actions sensibles	Planifie	Journalisation connexions, approbations
Droit d'accès aux données (Art. 15)	A implementer	Route GET /users/me/data-export
Droit a l'oubli (Art. 17)	A implementer	Route DELETE /users/me avec cascade
Portabilite des données (Art. 20)	A implementer	Export JSON/CSV profil + programmes
Politique de confidentialite	A implementer	Document legal avant ouverture
Hebergement EU	A verifier	Confirmer region S3 et serveurs dans UE

4. Audit d'accessibilite - WCAG 2.1 AA

L'accessibilite est un pilier fondateur de Coach'In. L'objectif est la conformite WCAG 2.1 niveau AA minimum, avec plusieurs criteres AAA atteints grace aux choix de palette.

4.1 Palette de couleurs - Ratios de contraste

Combinaison	Codes hex	Ratio	Niveau	Usage
Blanc / Navy principal	#F0EEE8 / #0A0F1E	14.8:1	AAA	Corps de texte
Or / Navy principal	#C9A84C / #0A0F1E	11.3:1	AAA	Accents, titres
Or clair / Navy	#E8C97A / #0A0F1E	11.1:1	AAA	Sous-titres
Muted / Navy	#8A96B0 / #0A0F1E	5.2:1	AA	Textes secondaires
Navy / Blanc	#0A0F1E / #F0EEE8	14.8:1	AAA	Boutons CTA

4.2 Implementation technique web (Next.js)

- React Aria (Adobe) : composants accessibles pour menus, modales, dialogs, tabs
- ESLint jsx-a11y : linting automatique des erreurs d'accessibilite dans le JSX
- Skip links : lien Aller au contenu principal sur toutes les pages
- ARIA landmarks : <header>, <main>, <nav>, <aside>, <footer> semantiques
- Focus visible : contour 2px minimum sur tous les elements interactifs
- aria-live regions : notifications dynamiques annoncees aux lecteurs d'ecran
- Attribut lang='fr' defini sur la balise HTML racine
- alt descriptifs : toutes les images et icones ont une alternative textuelle

4.3 Implementation technique mobile (Flutter)

- Widget Semantics : encapsule chaque widget interactif (label, hint, actions)
- Support TalkBack (Android) et VoiceOver (iOS) - tests systematiques
- Tailles cibles tactiles : minimum 48x48dp (depasse WCAG 44x44px recommandes)
- SemanticsService.announce() : annonces pour changements d'etat importants
- Respect prefers-reduced-motion : animations desactivables
- flutter_test avec semantics matchers : validation automatisee en CI

4.4 Pipeline de validation CI/CD

Outil	Type	Critere
axe DevTools	Automatise - Web	0 violation critique ou serieuse
Lighthouse a11y	Automatise - Web	Score >= 90 (bloque le merge), objectif >= 95
Pa11y (CI pipeline)	Automatise - CI/CD	Execution a chaque commit, rapport

Outil	Type	Critere
		automatique
NVDA + Chrome	Manuel - Windows	Navigation clavier complete, toutes les features
VoiceOver (iOS/macOS)	Manuel - Apple	Gestes et navigation vocale sur app mobile
TalkBack (Android)	Manuel - Android	Navigation accessible sur app mobile Flutter
Simulateur daltonisme	Manuel	Protanopie, deuteranopie, tritanopie

5. Performances et scalabilite

5.1 Optimisations frontend

- SSR Next.js : First Contentful Paint < 1.5s sur les pages catalogue coachs
- Code splitting : seul le bundle necessaire a chaque page est charge
- Images next/image : WebP automatique, lazy loading natif, srcset responsive
- Prefetching : liens visibles dans le viewport pre-charges automatiquement
- Compression Gzip/Brotli : activation nginx ou Vercel Edge Network

5.2 Optimisations backend

- Rate limiting NestJS : protection abus (100 req/min par IP par default)
- Indexes PostgreSQL : champs specialty, tags, isApproved, userId
- Pagination : toutes les listes avec limit/offset, jamais de SELECT * complet
- Projections SELECT : retourner uniquement les champs necessaires
- Connection pooling TypeORM : pool configure selon la charge cible

5.3 Seuils de performance cibles

Metrique	Objectif	Outil de mesure
Lighthouse Performance Score	>= 85	Lighthouse CI
First Contentful Paint (FCP)	< 1.5s	Core Web Vitals
Largest Contentful Paint (LCP)	< 2.5s	Core Web Vitals
Cumulative Layout Shift (CLS)	< 0.1	Core Web Vitals
Temps reponse API P95	< 200ms	Monitoring backend
Temps reponse API P99	< 500ms	Monitoring backend
Disponibilite (uptime)	>= 99.5%	Healthcheck automatise

6. Infrastructure et deployment

6.1 Architecture Docker

L'ensemble de la stack est containerisee avec Docker Compose. Chaque service tourne dans un conteneur isole avec son reseau et ses volumes persistes.

Services : app-web (Next.js:3000) | api (NestJS:3001) | db (PostgreSQL:5432) | nginx (reverse proxy:80/443). Volumes persistes pour PostgreSQL. Reseau interne isole entre les services.

- docker-compose.dev.yml / docker-compose.prod.yml : configs separees
- .env.dev / .env.prod : variables d'environnement par contexte
- Health checks sur chaque service avec restart: unless-stopped
- Volumes nommes pour persistance des donnees PostgreSQL

6.2 Pipeline CI/CD

- TypeScript check + ESLint : bloque le merge si erreurs
- Tests unitaires Jest : couverture minimum requise
- Tests accessibilite Pa11y + axe-core : 0 violation critique
- Build Docker : construction image de production
- Deploy staging : automatique sur merge main
- Deploy production : manuel apres validation staging

6.3 Stockage fichiers diplomes (S3)

- Upload via URL pre-signe depuis le frontend (expiration 15 min)
- Acces en lecture via URL signee (expiration 1h) pour l'admin uniquement
- Bucket prive : aucun acces public aux fichiers de diplomes
- Chiffrement at-rest AES-256 cote serveur
- Versioning S3 active pour conservation des versions precedentes

Verification requisite : Confirmer que le bucket S3 est dans une region EU (eu-west-3 Paris) pour conformite RGPD. Verifier la politique IAM : acces limite au role applicatif uniquement.

7. Etat actuel du projet

7.1 Tableau de bord des composants

Composant	Statut	Detail
Authentification JWT + cookies httpOnly	Operationnel	Login, inscription, deconnexion
Inscription athlete (2 etapes)	Operationnel	Tags, objectifs, profil complet
Inscription coach (3 etapes)	Operationnel	Specialty, bio, upload diplomes
Upload + stockage diplomes (S3)	Operationnel	FormData, workflow admin
Workflow admin approbation/refus	Operationnel	Preview PDF/image, approve/refuse
Profil athlete (onglets)	Operationnel	Vue ensemble, programmes mock, messages mock
Catalogue coaches + fiche profil	Operationnel	/coachinfo/[id], filtres tags
Demande de suivi athlete -> coach	Operationnel	POST /relationships, statuts
Authentification admin	Critique	Guard @Roles(admin) MANQUANT sur routes admin
PATCH /athletes/me (edit goals)	A implementer	handleSaveGoals est un stub TODO
Programmes reels (API)	A implementer	Actuellement MOCK_PROGRAMMES code en dur
Messagerie réelle	A implementer	Actuellement MOCK_MESSAGES code en dur
Profil coach (/profilcoach)	A implementer	Page a construire
Droits RGPD (export/suppression)	A implementer	Routes data-export et delete account
Notifications push Flutter	Planifie	Q4 2026 selon roadmap
OAuth Google	Planifie	Bouton present, logique non cabled